THEORETICAL PART

Student Record and Academic Management System

Student Name	Nikita Dnyaneshwar Udavant
PRN	2127UDSF1031
Subject	Python Programming

Data Organization using Python Collections

1. Objective

To apply Python built-in data structures to store, organize, update and retrieve student academic information.

2. Problem Analysis

A college needs a simple system to maintain student records. Each record may contain roll number, name, department, subjects, marks, attendance, email and address. The system should add, search, update, delete and display records, calculate average marks, find the highest scorer and list students by department.

Inputs and Outputs

Inputs: Roll number, name, department, subjects, marks, attendance and contact details.
Outputs: Student details, average marks, highest scorer, department-wise list and status messages.

3. Selection of Data Structures

Collection	Use
String	Stores text such as name, department, address and email.
List	Stores student records, subjects and marks.
Tuple	Keeps fixed information together, such as roll number and department.
Set	Stores unique department and subject names.
Dictionary	Stores one student's details as key-value pairs.

4. Algorithms

Add Student

Start → Enter details → Check duplicate roll number → If duplicate, show message; otherwise store record → Show success → Stop.

Search Student

Start → Enter roll number → Search each record → If found, display details → Otherwise show not found → Stop.

Update Student

Start → Enter roll number → Search record → If found, enter and save updated details → Otherwise show not found → Stop.

Delete Student

Start → Enter roll number → Search record → If found, remove record → Display result → Stop.

Calculate Average

Start → Select marks → Add marks → Divide total by number of marks → Display average → Stop.

Find Highest Scorer

Start → Compare each student's average → Keep highest average → Display student's name → Stop.

List by Department

Start → Enter department → Check all records → Display matching students → If none, show message → Stop.

5. Flowcharts

Add Student:

ADD STUDENT
START (Oval)
↓
Enter student details (Input)
↓
Roll number exists? (Decision)
↓
Yes → Show duplicate message
↓
No → Store record
↓
Show success message
↓
STOP (Oval)

Search Student:

SEARCH STUDENT
START (Oval)
↓
Enter roll number (Input)
↓
Search record (Process)
↓
Record found? (Decision)
↓
Yes → Display details
↓
No → Display not found
↓
STOP (Oval)

6. Pseudocode

```
ADD_STUDENT
START
INPUT roll_number
IF roll_number already exists
    PRINT "Duplicate roll number"
ELSE
    INPUT student details
    STORE student record
    PRINT "Student added"
END IF
STOP

SEARCH_STUDENT
START
INPUT roll_number
FOR each student in student list
    IF roll number matches
        DISPLAY student details
        STOP SEARCH
    END IF
END FOR
PRINT "Student not found"
STOP

CALCULATE_AVERAGE
START
total = 0
FOR each mark in marks
    total = total + mark
END FOR
average = total / number of marks
DISPLAY average
STOP
```

7. Testing and Test Cases

Test	Input	Expected result
Add student	New roll number	Record is added
Duplicate	Existing roll number	Duplicate message
Search	Existing roll number	Details displayed
Missing record	Unknown roll number	Not-found message
Average	70, 80, 90	Average = 80
Empty records	No students stored	No-records message

8. Observations

Lists store multiple records and marks. Dictionaries keep one student's information together. Tuples can store fixed values, and sets remove duplicates. Testing helps identify errors before use.

9. Conclusion

The project shows how Python collections can organize student information and support basic record operations.